



Week 8 Workshop Solutions

IFN647

School of Computer Science
Queensland University of Technology

Workshop Task 1 Solution

Algorithm *baseline_bm25*(Q, U)

Inputs: a query Q , a set of documents U in the “Traning_set” folder.

Output: a text file BaselineModel_R102.dat; where each row includes the document number and the corresponding ranking score (in descendent order).

Method:

Step 1. # Documents' representation

Let *docs* be an empty dictionary

For each xml document d in U : # text pre-processing

Get the document_ID

Find the contents in $\langle \text{text} \rangle \dots \langle / \text{text} \rangle$:

get tokens (or terms) and their frequencies, and

append (document_ID, {term:freq, ...}) into *docs*;

Step 2.

Calculate each term's document frequency and save the result in a dictionary *df_*

Step 3.

Use query Q , *docs* and *df_* to work out BM25 score for each document in U

Step 4

Sort the documents based on their BM25 score and save the result into a file
BaselineModel_R102.dat

Input

Output

Document representation
using term frequency

Term's Document
frequency

Calculate BM25

Sort the documents
on BM25

Workshop Task 1 Solution

Algorithm *baseline_bm25(Q, U)*

Inputs: a query Q , a set of documents U in the “Traning_set” folder.

Output: a text file BaselineModel_R102.dat; where each row includes the document number and the corresponding ranking score (in descendent order).

Method:

Step 1. # Documents' representation

Let *docs* be an empty dictionary

For each xml document d in U : # text pre-processing

Get the document_ID

Find the contents in `<text> ... </text>`:

get tokens (or terms) and their frequencies, and

append (document_ID, {term:freq, ...}) into *docs*;

Step 2.

Calculate each term's document frequency and save the result in a dictionary *df_*

Step 3.

Use query Q , *docs* and *df_* to work out BM25 score for each document in U

Step 4

Sort the documents based on their BM25 score and save the result into a file
BaselineModel_R102.dat

r_i is the number of relevant documents containing term i

n_i is the number of documents containing term i

N is the total number of documents in the collection

R is the number of relevant documents for this query

f_i is the frequency weight (count) of term i in the document

qf_i is the frequency weight (count) of term i in the query

$$\sum_{i \in Q} \log \frac{(r_i + 0.5) / (R - r_i + 0.5)}{(n_i - r_i + 0.5) / (N - n_i - R + r_i + 0.5)} \cdot \frac{(k_1 + 1) f_i}{K + f_i} \cdot \frac{(k_2 + 1) q f_i}{k_2 + q f_i}$$

Workshop Task 2 Solution

```
writeFile = open('PTraining_benchmark.txt', 'a')
bm25_threshold = 1.0
# you may change it to a different value, e.g., 5.0 for BaselineModel_R102_v2.dat

datFile = open('BaselineModel_R102.dat')
file_ = datFile.readlines()
for line in file_:
    line = line.strip()
    lineStr = line.split()
    if float(lineStr[1]) > bm25_threshold:
        writeFile.write('R102 ' + lineStr[0] + ' 1' + '\n')
    else:
        writeFile.write('R102 ' + lineStr[0] + ' 0' + '\n')
writeFile.close()
datFile.close()
```

Open the output file for appending, and create this file if it does not exist.

Set a threshold

Open the input file and read it as a list

For each line, remove its space at the beginning and ending and then split it with the whitespace separator.

If a document's BM25 score is greater than the threshold, give it a label, which is 1. Otherwise, it is marked as 0.

Input file: BaselineModel_R102.dat

```
73038 5.898798484774149
26061 4.273638903483098
65414 4.1414522450167475
57914 3.967136888209526
58476 3.708467957856744
76635 3.5867337114200843
12769 3.4341129093591456
12767 3.352170358051889
```

Output file: PTraining_benchmark.txt

```
R102 73038 1
R102 26061 1
R102 65414 1
R102 57914 1
R102 58476 1
R102 76635 1
R102 12769 1
R102 12767 1
R102 25096 1
R102 78836 1
...
R102 86912 0
R102 86929 0
R102 11922 0
```

Workshop Task 2 Solution

```
# Main function for Task 2

if __name__ == "__main__":

    import sys
    import os
    #import coll
    #import df

    writeFile = open('PTraining_benchmark.txt', 'a')
    bm25_threshold = 1.0
    # you may change it to a different value, e.g., 5.0 for BaselineModel_R102_v2.dat

    datFile = open('BaselineModel_R102.dat')
    file_ = datFile.readlines()
    for line in file_:
        line = line.strip()
        lineStr = line.split()
        if float(lineStr[1]) > bm25_threshold:
            writeFile.write('R102 ' + lineStr[0] + ' 1' + '\n')
        else:
            writeFile.write('R102 ' + lineStr[0] + ' 0' + '\n')
    writeFile.close()
    datFile.close()
```

Task 3 Sample Solutions

Refer week 7 solution code

```
def get_re_docs(inputpath):
```

```
    os.chdir(inputpath)
```

```
    A = {}
```

```
    B = {}
```

```
    for line in open("Training_benchmark.txt"):
        line = line.strip()
        line1 = line.split()
        A[line1[1]] = int(float(line1[2]))
```

```
    for line in open("PTraining_benchmark.txt"):
        line = line.strip()
        line1 = line.split()
        B[line1[1]] = int(float(line1[2]))
```

```
    return (A,B)
```

Relevant / Not relevant

Actual

Retrieved / Not retrieved

Given by pseudo relevance hypothesis

Training_benchmark.txt

```
R102 10182 0
R102 11083 0
R102 11481 1
R102 11485 1
R102 11922 1
R102 11923 1
R102 11930 1
R102 11960 0
R102 11979 1
R102 12479 1
```

PTraining_benchmark.txt

```
R102 73038 1
R102 26061 1
R102 65414 1
R102 57914 1
R102 58476 1
R102 76635 1
R102 12769 1
R102 12767 1
R102 25096 1
R102 78836 1
R102 82227 1
R102 26611 1
R102 24515 1
R102 33172 1
```

Read Training_benchmark.txt and save the pair of documentID and relevant judgement in dictionary A

Read PTraining_benchmark.txt and save the pair of documentID and retrieved judgement in dictionary B

Task 3 Sample Solutions **Refer week 7 solution code**

```
if __name__ == '__main__':
    import sys

    if len(sys.argv) != 2:
        sys.stderr.write("USAGE: %s <coll-file>\n" % sys.argv[0])
        sys.exit()
```

```
(rel_doc, retrived doc) = get_re_docs(sys.argv[1])
```

```
R = 0
for (x,y) in rel_doc.items():
    if (y==1):
        R= R+1
print("the number of relevant docs: " + str(R))
```

```
R1 = 0
for (x,y) in retrived_doc.items():
    if (y==1):
        R1= R1+1
print("the number of retrieved docs: "+ str(R1))
```

```
RR1 = 0
for (x,y) in rel_doc.items():
    if (y==1) & (retrived_doc[x]==1):
        RR1= RR1+1
print("the number of retrieved docs that are relevant: " + str(RR1))
```

```
r = float(RR1)/float(R)
p = float(RR1)/float(R1)
F1 = 2*p*r/(p+r)
print("recall = " + str(r))
print("precision = " + str(p))
print("F-Measure = " + str(F1))
```

$$Recall = \frac{|A \cap B|}{|A|} \quad \text{F-Measure (F1),}$$

$$Precision = \frac{|A \cap B|}{|B|} \quad \text{where } R \text{ is Recall, } P \text{ is Precision.}$$

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

Call the get_re_docs() function and get returned dictionaries, rel_doc and retrived_doc

Count the number of relevant documents based on the relevant judgement dictionary, rel_doc

Compute the number of retrieved documents based on the returned dictionary, retrived_doc

Get the number of retrieved relevant documents with two returned dictionaries

Calculate Recall, Precision and F-measure based on their equations

Workshop Task 3 Output and Discussion

the number of relevant docs: 106

the number of retrieved docs: 25

the number of retrieved docs that are relevant: 20

`recall = 0.18867924528301888`

`precision = 0.8`

$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$

Out of all actual relevant, how many were correctly predicted as relevant
20 / 106 (18%)

Out of all predicted as relevant, how many were actually relevant
20/25 (80%)

Please analyse the outputs to understand possible uncertainties (or errors) in the generated training set “PTraining_benchmark.txt”. You can also discuss any possible ways to generate a high-quality training set.

Solution 1:

Typically, there is a **trade-off between precision and recall**.

if you want to **increase the recall value**,
you can change the condition

"document's BM25 score is greater than 1.00" to

"document's BM25 score is greater than 0.80";

however, **precision may be reduced**.

Solution 2:

Also, you may ignore some indeterminate documents

ranked in middle, and label top-ranked

documents as “positive” and bottom-ranked documents

as “negative”.

Solution 3: New Idea ????

Workshop Task 3 Output and Discussion

the number of relevant docs: 106
the number of retrieved docs: 25
the number of retrieved docs that are relevant: 20

recall = 0.18867924528301888
precision = 0.8

$$\text{Recall} = \frac{|A \cap B|}{|A|}$$
$$\text{Precision} = \frac{|A \cap B|}{|B|}$$

Out of all actual relevant, how many were correctly predicted as relevant
20 / 106 (18%)

Out of all predicted as relevant, how many were actually relevant
20/25 (80%)

Please analyse the outputs to understand possible uncertainties (or errors) in the generated training set “PTraining_benchmark.txt”. You can also discuss any possible ways to generate a high-quality training set.

Solution 1:

Typically, there is a **trade-off between precision and recall**. if you want to **increase the recall value**, you can change the condition **"document's BM25 score is greater than 1.00"** to **"document's BM25 score is greater than 0.80"**; however, **precision may be reduced**.

Solution 2:

Also, you may ignore some indeterminate documents ranked in middle, and label top-ranked documents as “positive” and bottom-ranked documents as “negative”. This method **reduces noise** from uncertain documents.

Solution 3: New Idea ????

Collect more data

Use a different model or algorithm/ensemble methods, or LLMs